

Time: 2026-08-25 18:12

Author: Trần Xuân Bách

Status: #seed

Tags: Machine Learning

Theo đề xuất chỉ mentor thì em viết tay (trừ mấy đoạn em hỏi gpt xong paste vào :v) cả đống này kể cả equation =))

What is Text Classification

Có rất nhiều ứng dụng ví dụ như xác định đoạn văn nào do tác giả nào viết, người viết là nam hay nữ, spam mail, phân loại cảm xúc

Input

- ****Một tập hợp các phân lớp cố định $C=\{c_1, c_2, \dots, c_J\}$**
 - Đây là danh sách các danh mục hoặc nhãn (label) đã được định nghĩa sẵn. Đặc tính "cố định" ở đây đóng vai trò then chốt vì nó giới hạn không gian quyết định của mô hình; mô hình máy học chỉ được phép phân loại văn bản vào những nhóm đã được quy định trước chứ không được tự động sinh ra danh mục mới.

Output

- **Một phân lớp được dự đoán $c \in C$ (a predicted class $c \in C$)**
 - Sau khi phân tích tài liệu d , thuật toán sẽ tính toán và trả về một kết quả c duy nhất. Ký hiệu toán học $c \in C$ là một ràng buộc chặt chẽ, chỉ ra rằng nhãn được dự đoán c bắt buộc phải là một phần tử thuộc tập hợp C đã cung cấp ở đầu vào

Classification method

Cách tiếp cận dễ hiểu nhất là hard-coded rules

- Spam
 - Black list mail hoặc bắt chuỗi như "have been selected"

pros

- accuracy cao, tốc nhanh

cons

- thường là expert field
- không bắt được case chưa từng xuất hiện hoặc lách
 - ⇒ Vẫn giữ dùng kết hợp với supervised machine learning

Naive Bayes

For doc **d** and a class **c**

$$P(c|d) = \frac{P(d|c)P(c)}{P(d)}$$

Ta cần tìm max của

$$\hat{c} = \arg \max_{c \in C} P(c|d)$$

hay max của

$$\hat{c} = \arg \max_{c \in C} \frac{P(d|c)P(c)}{P(d)}$$

hay max của

$$\hat{c} = \arg \max_{c \in C} P(d|c)P(c)$$

cái probability của doc thì khá chắc là identical cho tất cả (const) nếu không nói là 1
nên ta bỏ qua

Note

- $P(c | d)$ là **Xác suất hậu nghiệm (Posterior probability)**: Xác suất văn bản d thuộc về phân lớp c
- $P(d | c)$ là **Hàm hợp lý (Likelihood)**: Xác suất sinh ra văn bản d nếu biết trước nó thuộc lớp c
- $P(c)$ là **Xác suất tiên nghiệm (Prior probability)**: Xác suất bắt gặp lớp c trong toàn bộ không gian mẫu (tập dữ liệu) mà chưa cần xét đến nội dung văn bản
- $P(d)$ là **Xác suất biên (Marginal probability)**: Xác suất xuất hiện của chính văn bản d
 - Nó đóng vai trò là một hằng số chuẩn hóa để đảm bảo tổng xác suất hậu nghiệm bằng 1

Multinomial

Bản chất của sự biến đổi từ $P(d | c)$ sang $P(x_1, x_2, \dots | c)$

Ký hiệu d (document) đại diện cho một văn bản nguyên khối. Tuy nhiên, các thuật toán học máy không thể "đọc" một đoạn văn bản theo cách con người hiểu ngữ nghĩa. Chúng cần chuyển đổi dữ liệu thô này thành các đặc trưng toán học

Quá trình này bẻ nhỏ văn bản d thành một chuỗi tuần tự các thành phần (thường là các từ hoặc cụm từ), được ký hiệu là các biến trạng thái $x_1, x_2, \dots, x_n$ (với n là tổng số từ trong văn bản). Do đó, việc tính xác suất để lớp c sinh ra văn bản d , tức là $P(d|c)$, về mặt bản chất chính là tính xác suất để lớp c sinh ra chuỗi các từ đó đi cùng nhau:

$$P(d | c) = P(x_1, x_2, \dots, x_n | c)$$

Ví dụ phân tích: Xét bài toán phân loại email rác (lớp $c=Spam$). Giả sử văn bản đầu vào là $d="Nhận thưởng tiền mặt"$. Hệ thống sẽ bẻ nhỏ d thành các vector đặc trưng:

- $x_1="Nhận"$
- $x_2="thưởng"$
- $x_3="tiền"$
- $x_4="mặt"$

Khi đó, hàm hợp lý $P(d|Spam)$ chính là việc đi tìm lời giải cho câu hỏi: Xác suất để 4 từ "Nhận", "thưởng", "tiền", "mặt" cùng xuất hiện trong một email rác là bao nhiêu, ký hiệu là $P("Nhận", "thưởng", "tiền", "mặt"|Spam)$

Hai giả định

Việc tính toán xác suất đồng thời của một chuỗi dài các biến $P(x_1, x_2, \dots, x_n)$

(c) trong thực tế là bất khả thi vì không gian trạng thái quá khổng lồ. Không có bất kỳ bộ dữ liệu nào đủ lớn để chứa mọi tổ hợp câu chữ có thể tồn tại trên đời. Để máy tính có thể giải quyết được, Naive Bayes bắt buộc phải áp đặt hai giả định nhằm đơn giản hóa bài toán toán học

Giả định thứ nhất: Mô hình Túi từ vụng (Bag of Words - Bỏ qua thứ tự) Giả định này tuyên bố rằng vị trí, cú pháp hay thứ tự của các từ trong câu hoàn toàn không mang lại giá trị thống kê. Hệ thống coi văn bản như một chiếc túi, trong đó các từ bị đổ lộn xộn vào nhau

- Quay lại ví dụ trên, đối với máy tính áp dụng giả định Bag of Words, hai câu "Nhận thưởng tiền mặt" và "Tiền mặt nhận thưởng" cung cấp lượng thông tin toán học chính xác như nhau. Hệ thống chỉ đếm tần suất (ví dụ: từ "tiền" xuất hiện 1 lần) chứ không quan tâm từ "tiền" đứng ở vị trí thứ nhất hay thứ ba. Việc bỏ qua thứ tự này phá vỡ hoàn toàn cấu trúc ngữ pháp, nhưng nó giúp chuẩn hóa độ dài của mọi văn bản về một không gian vector tần suất cố định để dễ tính toán

Giả định thứ hai: Tính độc lập có điều kiện (Conditional Independence) Đây là giả định cốt lõi tạo nên chữ "Naive" (Ngây thơ). Nó tuyên bố rằng: Khi đã biết trước văn bản thuộc phân lớp c , sự xuất hiện của một từ hoàn toàn không bị ảnh hưởng bởi sự xuất hiện của các từ xung quanh nó. Nghĩa là x_1 không liên quan gì đến x_2

- Trong ngôn ngữ học thực tế, điều này cực kỳ phi logic. Nếu từ x_2 là "thưởng", xác suất rất cao từ x_1 đứng liền trước nó là "Nhận" hoặc "Trúng". Các từ trong ngôn ngữ có tính phụ thuộc thống kê rất mạnh. Tuy nhiên, Naive Bayes phớt lờ hoàn toàn mối liên kết này

Nhờ việc giả vờ rằng các biến x độc lập hoàn toàn với nhau, định lý toán học cho phép biến đổi một xác suất đồng thời phức tạp thành một chuỗi các phép nhân đơn giản:

$$P(x_1, x_2, x_3, x_4 \mid Spam) = P(x_1 \mid Spam) \cdot P(x_2 \mid Spam) \cdot P(x_3 \mid Spam) \cdot P(x_4 \mid Spam)$$

Mặc dù hai giả định trên mang tính chất sai lệch hoàn toàn so với cách ngôn ngữ loài người hoạt động, nhưng khi xử lý trên lượng dữ liệu lớn, việc bóc tách thành các phép

nhân xác suất rồi rạc lại hoạt động cực kỳ hiệu quả, giúp mô hình nắm bắt được các "từ khóa trọng tâm" để dự đoán chính xác phân lớp của văn bản

Logspace

Do nhân nhiều xác suất lại với nhau nên dẫn đến hiện tượng số rất nhỏ và tràn số thực
⇒ Chuyển phép nhân thành phép cộng

- Theo định lý cơ bản, logarit của một tích bằng tổng các logarit thành phần: $\log(A \cdot B) = \log(A) + \log(B)$
- Tính đồng biến (Monotonicity): Hàm logarit là một hàm đồng biến (strictly increasing function). Điều này có nghĩa là, nếu một giá trị A lớn hơn giá trị B, thì $\log(A)$ cũng chắc chắn lớn hơn $\log(B)$
⇒ Mục tiêu tối hậu của thuật toán Naive Bayes không phải là tính ra con số xác suất tuyệt đối chính xác là bao nhiêu, mà là đi tìm phân lớp c mang lại giá trị tương đối lớn nhất so với các lớp còn lại. Vì hàm logarit bảo toàn nguyên vẹn thứ tự lớn bé, phân lớp nào tối ưu hóa được phương trình phép nhân gốc thì cũng chính là phân lớp tối ưu hóa được phương trình phép cộng logarit. Quyết định phân loại cuối cùng hoàn toàn không bị thay đổi

$$\hat{c}_{NB} = \arg \max_{c \in C} \left(\log P(c) + \sum_{i=1}^n \log P(x_i | c) \right)$$

Prob về 0

Ví dụ khi trong câu negative không có chữ "lunch" thì prob nó sẽ = 0 vậy sẽ luôn là positive

→ để xử lý vấn đề này với mỗi từ hay token ta luôn cộng thêm 1 gọi là alpha

- Nhớ cộng trả lại tổng trong từ

word occurrence and frequency

- For task like sentiment occurrence seem to be more important than frequency

Binary Multinomial naive bayes

Lúc train khi 1 doc có 1 từ lặp lại thì ta drop về 1 thôi

Test data

- Gặp từ chưa xuất hiện trong từ điển → drop luôn

Vấn đề stopwords

- Không có easch lăm trong navie bayes text classification nên thường giữ lại hết

Dealing with negation

Cách đơn giản nhất là thêm NOT_ vào toàn bộ

Ex: Tôi không thích bộ phim này, nhưng mà

→ NOT_thích NOT_bộ NOT_phim NOT_này, nhưng mà

Dừng ở dấu câu tiếp

→ Áp dụng cách này sẽ làm phình to từ điển

Lexicon

Trong bài toán Phân tích Cảm xúc (Sentiment Analysis), Lexicon (hay Từ điển Cảm xúc) là một kho dữ liệu bao gồm các từ vựng đã được gán sẵn các giá trị phân cực cảm xúc (polarity). Quá trình gán nhãn này thường được thực hiện bởi các chuyên gia ngôn ngữ học

Mỗi từ trong Lexicon sẽ được ánh xạ với một trong hai dạng:

- **Phân loại nhãn:** Tích cực (Positive), Tiêu cực (Negative), Trung tính (Neutral).
- **Trọng số điểm (Scoring):** Các giá trị số học thể hiện cường độ, ví dụ từ "tốt" có điểm +1, nhưng từ "xuất sắc" có điểm +3; ngược lại "tệ" là -1 và "thảm họa" là -3.

Khi nào sử dụng Lexicon?

Việc sử dụng Lexicon đặc biệt phát huy tác dụng trong các kịch bản sau:

- **Không có dữ liệu huấn luyện (Unsupervised / Zero-shot):** Khi bạn cần phân loại văn bản nhưng không có sẵn một bộ dữ liệu khổng lồ đã được gán nhãn để huấn luyện các thuật toán máy học. Phương pháp Lexicon-based sẽ duyệt qua văn bản, cộng gộp điểm số của các từ xuất hiện trong từ điển và đưa ra kết luận cuối cùng (tổng điểm > 0 là tích cực)
- **Khai phá đặc trưng (Feature Extraction):** Trong các mô hình học máy cơ bản như Naive Bayes hay SVM, điểm số tính toán từ Lexicon có thể được sử dụng như một biến đầu vào (feature) bổ sung. Việc kết hợp này cung cấp cho mô hình một nền tảng kiến thức ngữ nghĩa tiên nghiệm, giúp tăng độ chính xác khi dữ liệu huấn luyện bị thiếu

Support-vector machines

Tên gọi "**support vector**" (**vector hỗ trợ**) xuất phát từ chính vai trò của các điểm dữ liệu đặc biệt nằm gần ranh giới phân tách nhất, là những điểm "chống đỡ" hoặc "định hình" nên vị trí của siêu phẳng (hyperplane) phân loại.

Nguồn gốc chi tiết của tên gọi:

- **Là các điểm dữ liệu (Vectors):** Trong không gian toán học, mỗi điểm dữ liệu được biểu diễn dưới dạng một vector tọa độ.
- **Có tính chất hỗ trợ (Support):** Chỉ một số ít điểm dữ liệu nằm sát đường ranh giới nhất (gọi là các vector hỗ trợ) mới quyết định toàn bộ vị trí và hướng của mô hình phân lớp. Nếu bạn bỏ đi tất cả các điểm dữ liệu khác và chỉ giữ lại các vector hỗ trợ này, ranh giới phân tách vẫn giữ nguyên vị trí cũ. Chúng "hỗ trợ" hoặc "chống đỡ" cho toàn bộ cấu trúc của ranh giới quyết định đó

Vấn đề dễ nhầm

- Bài toán luôn là hoặc A hoặc B nếu có thêm C vào thay vì phải chạy từng cặp hay OvO
 - thì thành A vs B gộp C, C vs A gộp B, B vs A gộp C

OvR là gì, khác kernel-based (OvO) chỗ nào

OvR (One-vs-Rest): với 3 lớp, train 3 classifier nhị phân, mỗi cái so 1 lớp với phần còn lại gộp chung:

```
clf1: Negative vs {Neutral, Positive}
clf2: Neutral vs {Negative, Positive}
clf3: Positive vs {Negative, Neutral}
```

Predict: tính 3 decision score, chọn lớp có score cao nhất.

OvO (One-vs-One): train từng cặp lớp riêng, bỏ qua lớp còn lại:

```
clf1: Negative vs Neutral
clf2: Negative vs Positive
clf3: Neutral vs Positive
```

Vấn đề chiều

document-term matrix

↓

shape = (số mẫu, số chiều) = (N, D)

N = số hàng = số document (samples)

D = số cột = số từ riêng biệt trong vocabulary (dimensions)

- **Số chiều (D)** = kích thước vocabulary = số từ/n-gram riêng biệt mà `TfidfVectorizer` học được từ toàn bộ tập dữ liệu. Đây là con số cố định cho toàn bộ bài toán — mọi document, dù dài hay ngắn, đều được biểu diễn bằng đúng D chiều đó (document nào không chứa 1 từ nào đó thì giá trị ở chiều tương ứng đơn giản là 0).
- **Số mẫu (N)** = số document = số hàng trong bảng. Đây là số lượng *điểm* mà bạn có trong không gian D chiều đó.

Khái niệm kernel

Evaluation

Confusion matrix — nền tảng của mọi metric

Trước khi vào từng metric, cần 4 khái niệm gốc, dùng ví dụ nhỏ: 10 email, model đoán "spam hay không spam".

	Model đoán Spam	Model đoán Không-spam
Thực tế là Spam	TP = 3	FN = 1
Thực tế Không-spam	FP = 2	TN = 4

- **TP (True Positive)**: đoán Spam, thực tế đúng là Spam → 3
- **TN (True Negative)**: đoán Không-spam, thực tế đúng là Không-spam → 4
- **FP (False Positive)**: đoán Spam, nhưng thực tế Không-spam (**báo động giả**) → 2
- **FN (False Negative)**: đoán Không-spam, nhưng thực tế là Spam (**bỏ sót**) → 1

Toàn bộ 4 metric bên dưới chỉ là các cách kết hợp khác nhau của 4 con số này.

Accuracy — đúng bao nhiêu trên tổng số

$$\text{Accuracy} = (\text{TP} + \text{TN}) / \text{tổng số} = (3 + 4) / 10 = 0.7 = 70\%$$

Ý nghĩa: đơn giản nhất — trong toàn bộ mẫu, đoán đúng bao nhiêu phần trăm.

Khi nào dùng: chỉ nên dùng khi các lớp **cân bằng** (số lượng mẫu mỗi lớp xấp xỉ nhau).

Khi nào KHÔNG nên dùng — vấn đề kinh điển: giả sử 1000 email, chỉ 10 cái là spam thật (990 không-spam). Một model "ngu" nhất, cứ đoán **tất cả đều là không-spam**, vẫn đạt:

$$\text{Accuracy} = 990/1000 = 99\%$$

Cao ngất ngưỡng nhưng model này **vô dụng hoàn toàn** — nó không bắt được 1 email spam nào. Đây chính là lý do trong bài của bạn (corpus tin tức với Neutral chỉ chiếm ~29%, Negative ~36%), accuracy dễ bị lệch bởi lớp đông hơn, và đây là lý do dùng `average="macro"` thay vì accuracy thuần.

Precision — trong số tôi *nói* là Positive, đúng bao nhiêu

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) = 3 / (3 + 2) = 0.6 = 60\%$$

Ý nghĩa: trong 5 email model gắn nhãn "Spam", chỉ 3 cái thực sự là spam, 2 cái bị gắn oan (báo động giả).

Khi nào ưu tiên precision: khi cái giá của báo động giả (FP) rất đắt. Ví dụ: hệ thống lọc spam đưa email quan trọng của sếp vào thùng rác (FP) — hậu quả nghiêm trọng hơn nhiều so với việc để lọt 1 email spam vào hộp thư (FN). Một ví dụ khác: hệ thống gợi ý "sa thải nhân viên vì gian lận" — báo động giả (đổ oan người vô tội) tốn kém hơn nhiều so với bỏ sót.

Recall — trong số thực sự Positive, tôi *bắt được* bao nhiêu

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = 3 / (3 + 1) = 0.75 = 75\%$$

Ý nghĩa: trong 4 email spam thực sự tồn tại, model chỉ bắt được 3, bỏ sót 1.

Khi nào ưu tiên recall: khi cái giá của bỏ sót (FN) rất đắt. Ví dụ kinh điển: chẩn đoán ung thư — bỏ sót 1 ca bệnh thật (FN) nguy hiểm hơn nhiều so với báo động giả 1 ca (chỉ tốn thêm 1 lần xét nghiệm kiểm tra lại). Recall cao đảm bảo "quét sạch", ít bỏ lọt.

F1 — cân bằng cả 2, không thiên vị bên nào

$$\begin{aligned} \text{F1} &= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \\ &= 2 \times (0.6 \times 0.75) / (0.6 + 0.75) \\ &= 2 \times 0.45 / 1.35 \\ &= 0.667 = 66.7\% \end{aligned}$$

Đây là **trung bình điều hòa** (harmonic mean), không phải trung bình cộng thường. Điểm đặc biệt của trung bình điều hòa: nó bị "kéo mạnh" về phía số nhỏ hơn — nếu 1 trong 2 (precision hoặc recall) quá thấp, F1 cũng thấp theo, dù số còn lại có cao đến đâu.

Ví dụ minh họa sự khác biệt: Precision = 0.9, Recall = 0.1

- Trung bình cộng: $(0.9+0.1)/2 = 0.5$ (trông có vẻ ổn)
- F1 (điều hòa): $2 \times 0.9 \times 0.1 / (0.9 + 0.1) = 0.18$ (phản ánh đúng là model đang tệ)

Khi nào dùng F1: khi bạn cần **cả precision lẫn recall đều ổn**, không muốn model "ăn gian" bằng cách hy sinh cái này lấy cái kia (ví dụ model cứ đoán tất cả là Positive thì recall = 100% nhưng precision rất thấp — F1 sẽ vạch trần điều đó, còn nếu chỉ nhìn recall thì bị lừa). Đây cũng là lý do dùng `f1_macro` trong bài 3 lớp của bạn.

Mẹo ghi nhớ — chỗ hay nhầm nhất là Precision vs Recall

Nhìn vào mẫu số — đây là cách phân biệt chắc chắn nhất:

```
Precision = TP / (TP + FP)  ← mẫu số là "tất cả những gì TÔI ĐÃ ĐOÁN là Positive"
Recall     = TP / (TP + FN)  ← mẫu số là "tất cả những gì THỰC SỰ là Positive"
```

Câu thần chú ngắn:

- **Precision = hỏi về lời tôi nói** ("Trong những gì tôi *khẳng định* là đúng, tôi đúng bao nhiêu?") → liên quan **FP** (khẳng định sai)
- **Recall = hỏi về thực tế** ("Trong tất cả những gì *thực sự tồn tại*, tôi tìm ra được bao nhiêu?") → liên quan **FN** (bỏ sót)

Một mẹo tiếng Anh hay dùng: "**Precision is about your Predictions, Recall is about Reality.**" — cả 2 chữ cái đầu (P-Predictions, R-Reality) trùng với tên metric, dễ nhớ.

Hoặc hình dung bằng ví dụ đời thường: bạn thả lưới bắt cá trong hồ có cả cá và rác.

- **Precision cao** = mở lưới ra, hầu hết đều là cá thật (ít rác lẫn vào) — quan tâm "trong mẻ lưới, cái gì cũng đúng là cá".
- **Recall cao** = bạn vét được gần hết cá trong hồ, không để sót con nào — quan tâm "trong cả hồ cá, mẻ lưới bắt được bao nhiêu phần".

Có thể lưới toàn rác (recall cao, precision thấp — nếu bạn quăng lưới quá to, bắt hết mọi thứ kể cả rác) **hoặc lưới sạch nhưng chỉ bắt được vài con** (precision cao, recall thấp — nếu bạn chỉ vớt cẩn thận từng con chắc chắn là cá, bỏ lỡ nhiều con khác) — 2 chỉ số này thường **đánh đổi lẫn nhau** (trade-off), và F1 là cách đo xem bạn cân bằng được 2 thứ đó tốt đến đâu.

References

[Naive Bayes, Clearly Explained!!!](#)

[5 1 What is Text Classification 8 12](#)

[Naive Bayes Lecture 2 The Naive Bayes Classifier](#)

[Naive Bayes 3 Learning in Naive Bayes](#)

[Naive Bayes 4 Sentiment and Binary NB](#)

[4 5 More on Sentiment Classification](#)

[5 7 Precision, Recall, and the F measure 16 16](#)

[Machine Learning Fundamentals: Bias and Variance](#)

[Machine Learning Fundamentals: Cross Validation](#)

[Support Vector Machines Part 1 \(of 3\): Main Ideas!!!](#)

[Support Vector Machines Part 2: The Polynomial Kernel \(Part 2 of 3\)](#)